BTS SERVICES INFORMATIQUES AUX ORGANISATIONS **SESSION 2026**

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (recto)

Épreuve E6 - Conception et développement d'applications (option SLAM)

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE	**N° réalisation : 2**
Nom, prénom : Chattou Imad	**N° candidat : 090589257BD**
Épreuve ponctuelle ☐ **Contrôle en cours de formation** ☒	**Date :** 17 / 03 /2026

Organisation support de la réalisation professionnelle
Projet réalisé en autonomie dans le cadre d'une mise en situation professionnelle au sein d'une agence web fictive. En tant que développeur web junior, j'ai conçu et développé seul une application web pour le comité d'organisation des JO de Los Angeles 2028, en suivant les phases classiques d'un projet : analyse des besoins, modélisation de la base de données, développement et tests.

Intitulé de la réalisation professionnelle
E6 6 Cartes grises - JAVA – Support étudiants

Période de réalisation : 02/12/2025 au 16/01/2026 **Lieu :** Lycée La Tournelle (La Garenne-Colombes)
Modalité : ☒ **Seul(e)** ☐ **En équipe**

Compétences travaillées
- ☒ Concevoir et développer une solution applicative
- ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative
- ☒ Gérer les données

Conditions de réalisation[5] (ressources fournies, résultats attendus)
Ressources fournies : Un cahier des charges fonctionnelles décrivant les besoins du client, des maquettes et une base de données relationnelle pré-établie.
Résultats attendus : un site web fonctionnel permettant la consultation des informations des JO 2028, ainsi qu'un espace administrateur pour la gestion des données.

Description des ressources documentaires, matérielles et logicielles utilisées[6]

Ressources documentaires
- Fiches ressources (cours) et supports étudiants (TP)
- Sites de ressources en ligne : Youtube

Ressources matérielles
- Ordinateur portable
- Connexion internet

Ressources logicielles
- IDE : Visual Studio Code
- Ensemble de solutions logicielles : MAMP (serveur Apache et base de données relationnelles MySQL)
- Outil de modélisation de base de données : Mocodo
- Outil de modélisation UML : Visual Paradigm Online
- Outil de gestion de projet : Trello

Langages
- Langages web : JAVA
- Langage base de données : SQL

Modalités d'accès aux productions[7] et à leur documentation[8]
- Cahier des charges (GitHub) : https://urlr.me/jW4Nds
- Code (GitHub) : https://urlr.me/x8GHv6

---

[5] En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

[6] Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

[7] Conformément au référentiel du BTS SIO « *Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve.* ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

[8] Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS **SESSION 2026**

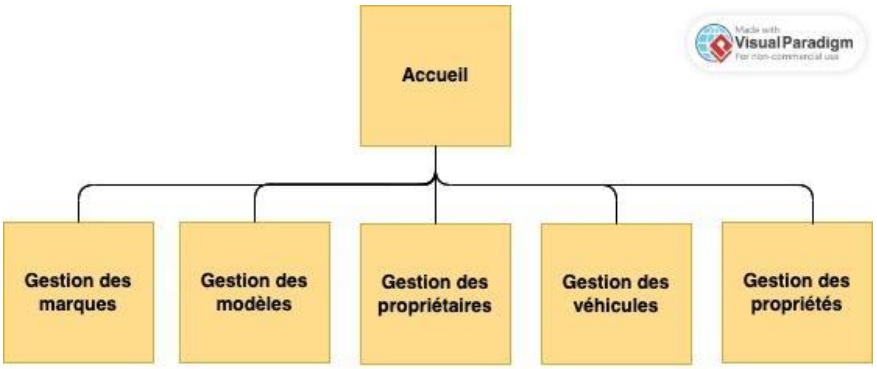
ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (verso, éventuellement pages suivantes)

Épreuve E6 - Conception et développement d'applications (option SLAM)

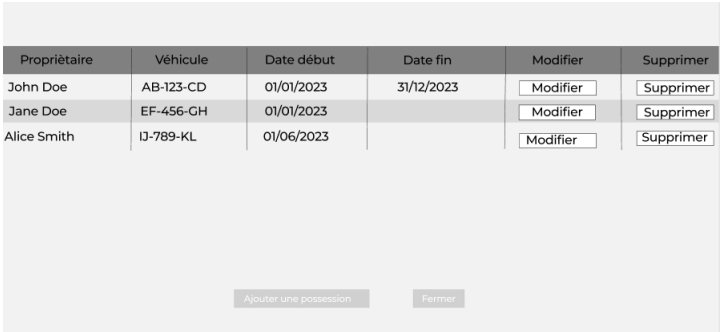
Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

L'objectif est de concevoir et développer un logiciel de gestion des cartes grises (ajout, modification et suppression d'information). Le logiciel est développé en Java

Arborescence

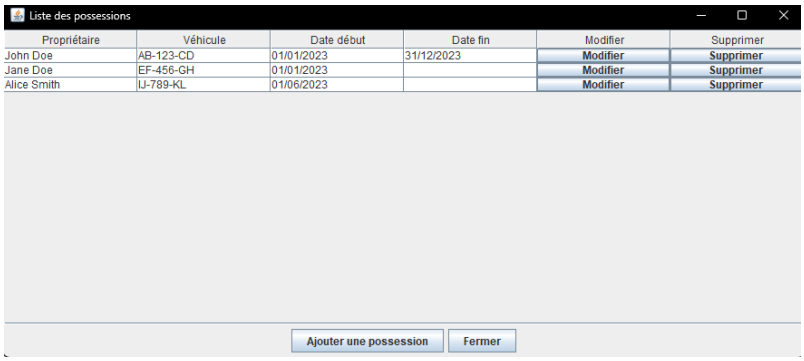


Wireframe



Wireframe de la page Possession, illustrant la structure et l'organisation de l'interface utilisateur.

Maquette

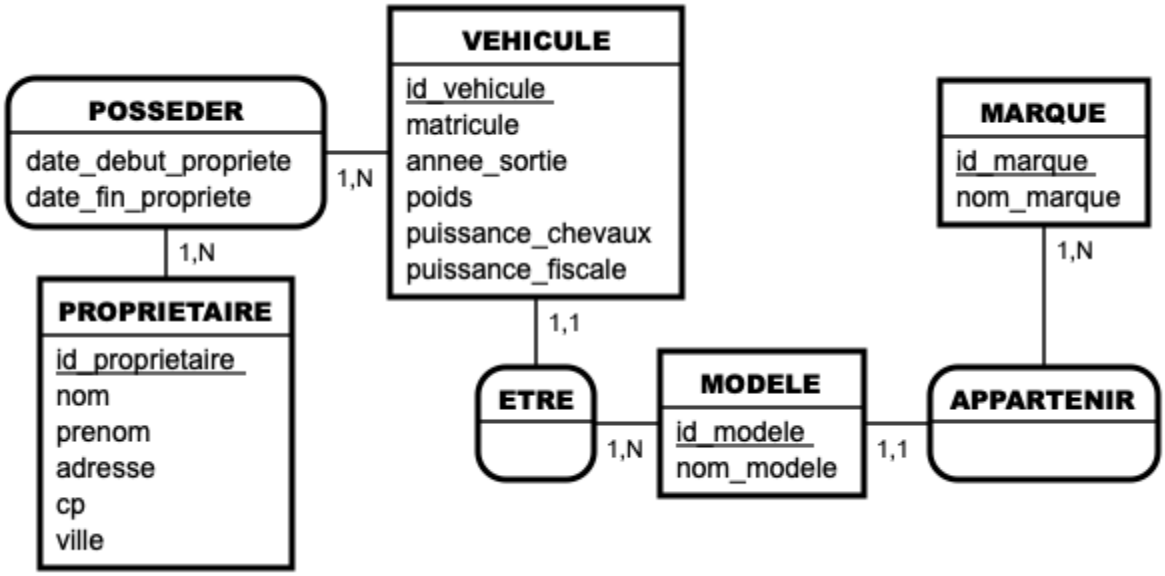


Maquette de l'interface de gestion des possessions, présentant la liste des propriétaires, véhicules associés et périodes de possession, avec des options de modification et de suppression.

Dictionnaire de données

Entité ou association	Libellé de l'attribut	Type
MARQUE	id_marque	INT(11)
/	nom_marque	VARCHAR(255)
MODELE	id_modele	INT(11)
/	nom_modele	VARCHAR(255)
POSSEDER	id_proprietaire	INT(11)
/	id_vehicule	INT(11)
/	date_debut_propriete	DATE
/	date_fin_propriete	DATE
PROPRIETAIRE	id_proprietaire	INT(11)
/	Nom	VARCHAR(255)
/	prenom	VARCHAR(255)
/	adresse	VARCHAR(255)
/	cp	VARCHAR(10)
/	ville	VARCHAR(255)
VEHICULE	id_vehicule	INT(11)
/	matricule	VARCHAR(50)
/	annee_sortie	YEAR(4)
/	poids	INT(11)
/	puissance_chevaux	INT(11)
/	puissance_fiscale	INT(11)

Modèle Conceptuel de données (MCD)



Modèle Logique de données (MLD)

- **MARQUE (id_marque, nom_marque)**
  Clé primaire :id_marque

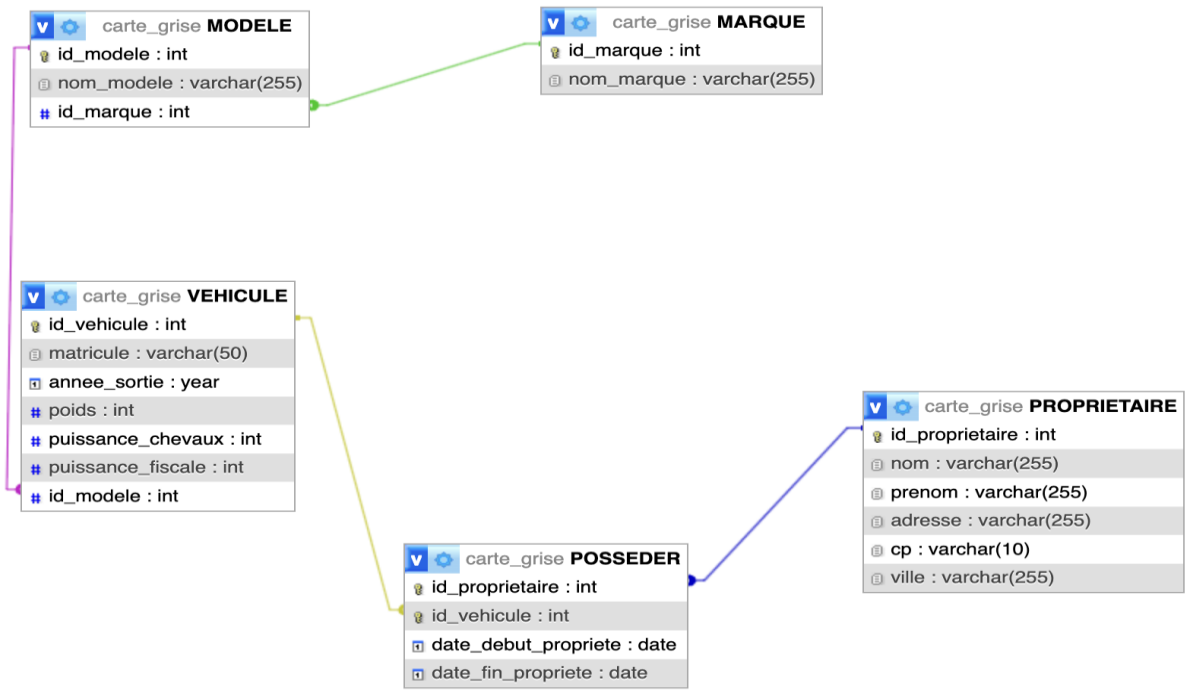
- **MODELE (id_modele, nom_modele, id_marque)**
  Clé primaire :id_modele
  Clé étrangère :id_marque en référence à id_marque de MARQUE

- **VEHICULE (id_vehicule, matricule, annee_sortie, poids, puissance_chevaux, puissance_fiscale, id_modele)**
  Clé primaire : id_vehicule
  Clé étrangère :id_modele en référence à id_modele de MODELE

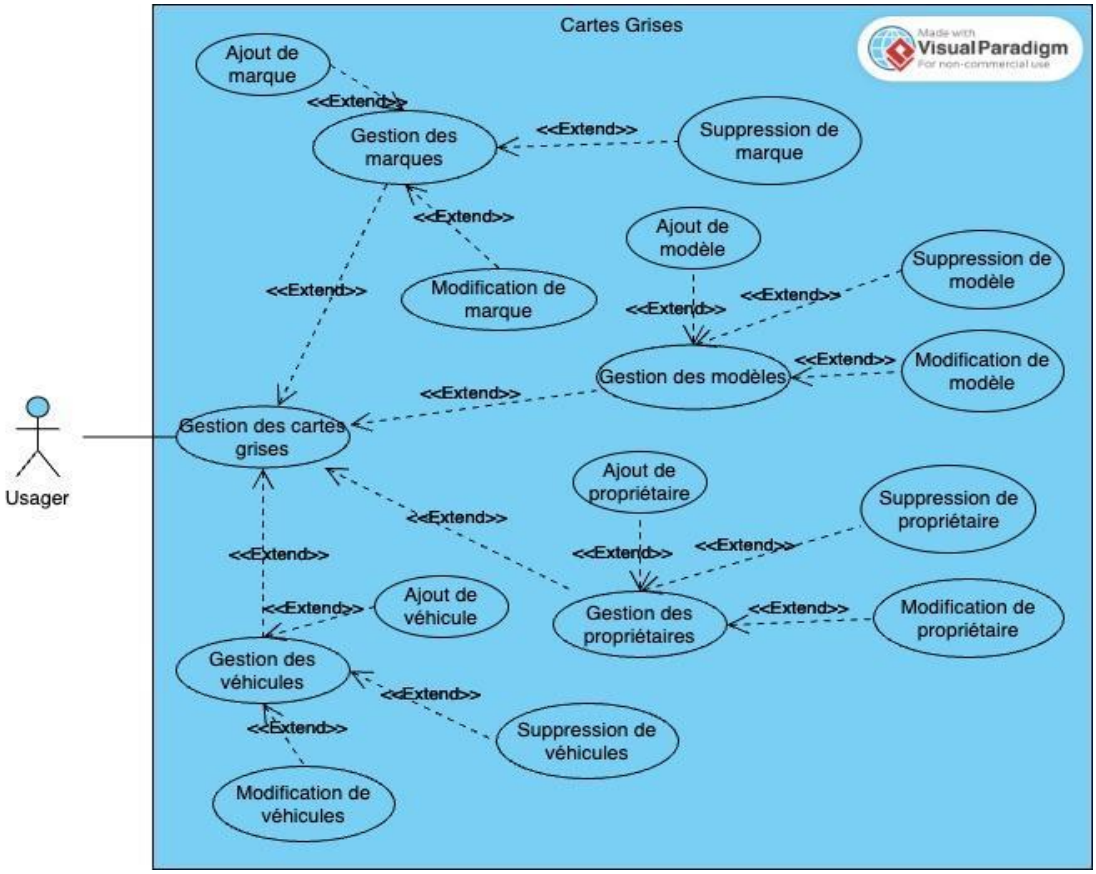
- **PROPRIETAIRE (id_proprietaire, nom, prenom, adresse, cp, ville)**
  Clé primaire :id_proprietaire

- **POSSEDER (id_proprietaire, id_vehicule, date_debut_propriete, date_fin_propriete)**
  Clé primaire :id_proprietaire, id_vehicule
  Clés étrangères :id_proprietaire en référence à id_proprietaire de PROPRIETAIRE id_vehicule en référence à id_vehicule de VEHICULE

Modèle Physique de données (MPD)

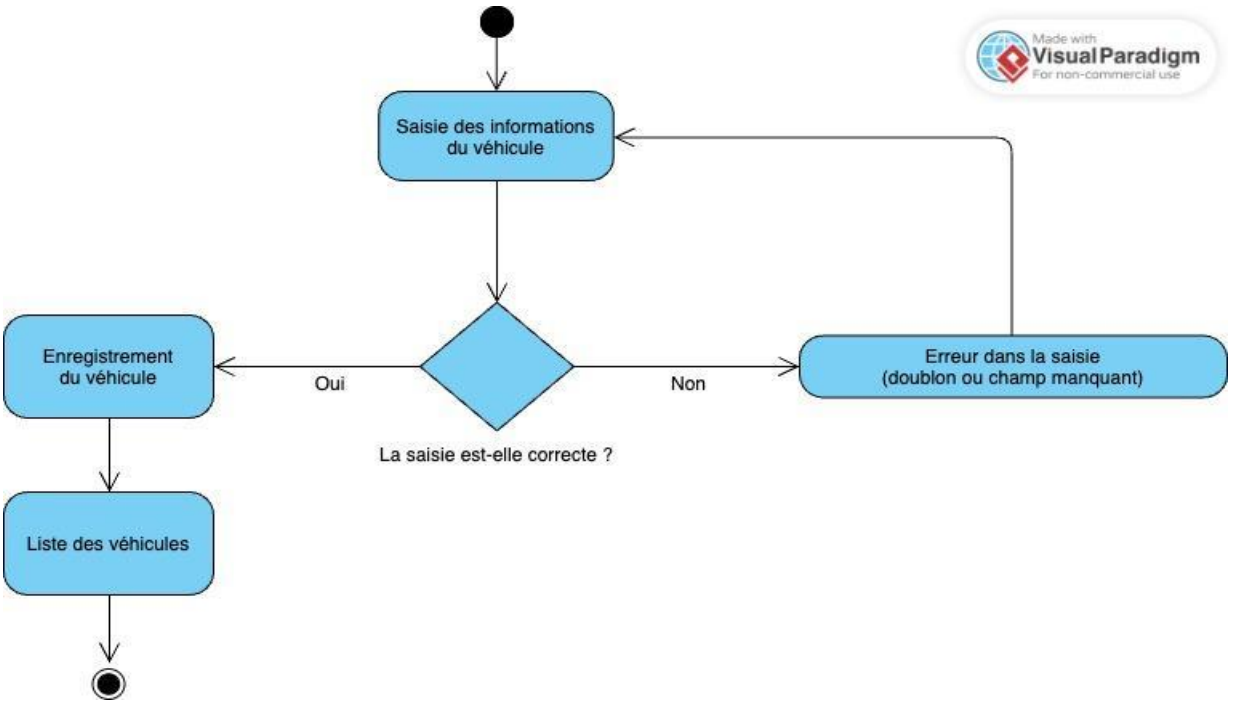


UML (diagramme de cas d'utilisations)

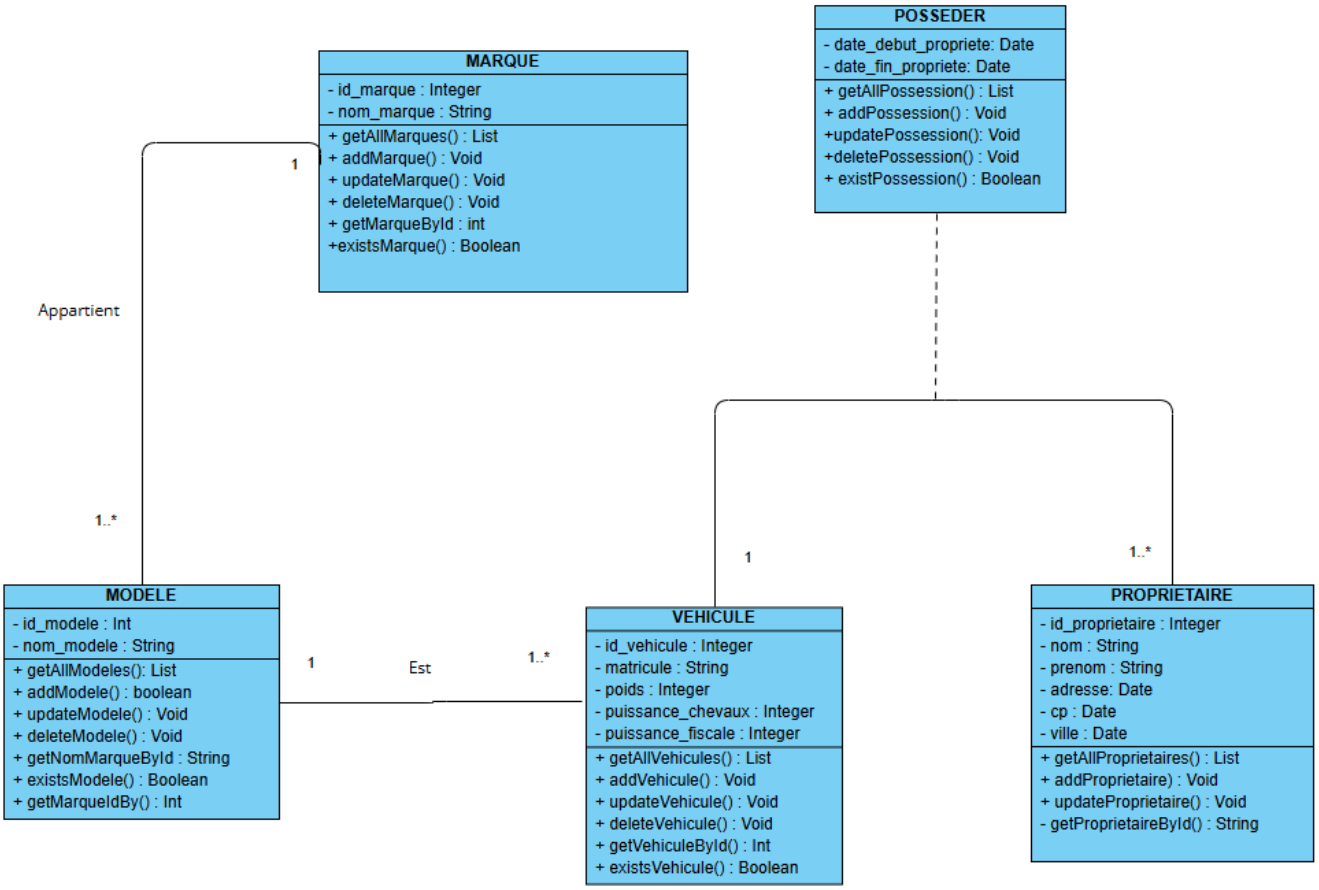


ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle

UML (diagramme d'activités)



UML (diagramme de classe)



Extrait de code pertinent

Code 1 :

- Cette méthode vérifie si un modèle existe déjà dans la base en comptant les enregistrements ayant le même nom et la même marque, avec la possibilité d'exclure un ID lors d'une modification pour éviter de se comparer à lui-même.
- Elle exécute la requête de façon sécurisée avec un PreparedStatement, récupère le résultat via un ResultSet, et retourne true si un doublon est trouvé, sinon false (avec gestion des erreurs SQL).

```
    /** Vérifier si un modèle existe (pour éviter les doublons) */

    public static boolean exists(String nom, int idMarque, Integer excludeId)
{

        String sql = "SELECT COUNT(*) FROM MODELE WHERE nom_modele = ? AND
id_marque = ?";

        if (excludeId != null) sql += " AND id_modele != ?";


        try (Connection conn = DBConnection.getConnection();

             PreparedStatement ps = conn.prepareStatement(sql)) {


            ps.setString(1, nom);

            ps.setInt(2, idMarque);

            if (excludeId != null) ps.setInt(3, excludeId);


            try (ResultSet rs = ps.executeQuery()) {

                if (rs.next()) return rs.getInt(1) > 0;

            }

        } catch (SQLException e) {

            System.err.println("Erreur exists : " + e.getMessage());

        }

        return false;

    }
```

Code 2:

- Cette méthode est appelée lorsqu'un bouton du tableau est utilisé : si c'est "Supprimer", elle demande une confirmation puis supprime le propriétaire et recharge le tableau ; si c'est "Modifier", elle récupère les données du propriétaire et ouvre le formulaire de modification.
- Elle utilise l'ID récupéré dans la ligne sélectionnée pour appeler le contrôleur, affiche un message selon le résultat, puis réinitialise l'état (clicked = false) avant de retourner le label du bouton.

```
        @Override

        public Object getCellEditorValue() {

            if (clicked) {

                int id = (int) table.getValueAt(row, 0);

                if (label.equals("Supprimer")) {

                    int confirm =
JOptionPane.showConfirmDialog(ProprietaireView.this,

                            "Supprimer ce propriétaire ?", "Confirmer",
JOptionPane.YES_NO_OPTION);

                    if (confirm == JOptionPane.YES_OPTION) {

                        boolean success =
proprietaireController.removeProprietaire(id);

                        JOptionPane.showMessageDialog(ProprietaireView.this,
success ? "Supprimé !" : "Erreur !");

                        refreshTable();

                    }

                } else if (label.equals("Modifier")) {

                    Proprietaire p =
proprietaireController.findProprietaireById(id);

                    if (p != null) showProprietaireForm(p);

                }

            }

            clicked = false;

            return label;

        }
```